



University of St.Gallen



University
of Otago
OTĀKOU WHAKAIHU WAKA

Towards Operationalizing Accountability for Self-Improving Multi-Agent Systems

Andrei Ciortea^{1,2} and Stephen Cranefield³

¹University of St.Gallen, Switzerland

²Inria, Sophia Antipolis, France

³University of Otago, New Zealand

EMAS 2026 — Paphos, 26 May 2026

From insight to impact.

Self-Improving Accountable Agents

Accountability is an **asymmetric power relationship** between an **accountee** (account taker) and one or more **accountors** (account givers) [Cranefield and Oren, 2026]



Accountor

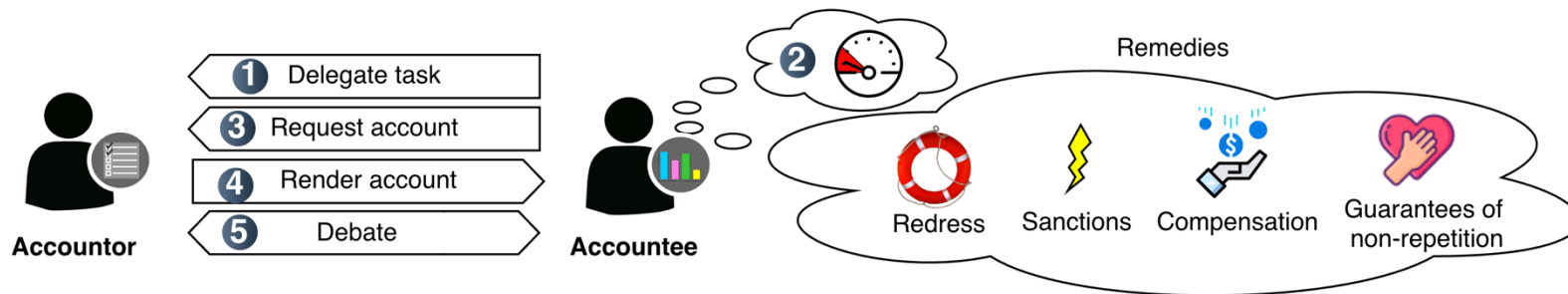


Accountee

Self-Improving Accountable Agents

Accountability is an **asymmetric power relationship** between an **accountee** (account taker) and one or more **accountors** (account givers) [Cranefield and Oren, 2026]

An **accountor** must be able to (i) **render accounts** and (ii) **learn** from the accountee's feedback



Our focus: cognitive agents equipped with procedural knowledge

Empirical exploration — we use LLMs to:

- render accounts
- analyze accounts and derive actionable feedback
- revise procedural knowledge based on feedback

Scenario: Home Heating



Rendering Accounts

An **account** is a collection of entries recorded at specific decision points relevant to a given task

Account Entry

- **metadata**: general information about the decision
- **decision**: concise description of what was decided
- **beliefs**: information held by the agent at decision time
- **options**: alternatives available at the time (if any)
- **initial env. state**: relevant environment state at decision time
- **current env. state**: relevant environment state at account time
- **skills**: skills that supported or influenced the decision

can be outsourced to
an external service

(the rest can be retrieved from
short- and long-term memory)

The account **traces** the accountant's
reasoning and context at decision time

An **accountable agent** proactively maintains the
information required to render accounts

Learning from Accounts

1. Account Analysis

Accountee uses an LLM as judge to analyze accounts

LLM Judge Evaluation Structure

- **root cause:** what caused the problem
- **blindspots:** what was ignored or overlooked
- **corrective actions:** fix the current situation
- **preventive actions:** avoid future occurrences

"rootCause": "The agent's heating plan (+temp(T) : target_temp(Target) & T < Target & heating(false)) only checks if the temperature is below target and heating is off, but does not consider the window state. The agent started heating while the window was tilted, which causes heat loss and makes the heating inefficient. The plan context is missing a check for window_state to ensure the window is closed before starting the heater.",

2. Skill Revision

Learning from Accounts

1. Account Analysis

Accountee uses an LLM as judge to analyze accounts

LLM Judge Evaluation Structure

- **root cause:** what caused the problem
- **blindspots:** what was ignored or overlooked
- **corrective actions:** fix the current situation
- **preventive actions:** avoid future occurrences

```
"correctiveActions": [  
  "Close the window immediately — since heating is now active (heating(true)) and the window  
    is still tilted (window_state(\"tilted\")), the agent should close the window to  
    prevent heat loss and improve heating efficiency .",  
  "If closing the window is not possible, the agent should stop heating to avoid wasting  
    energy until the window can be closed."  
],
```

2. Skill Revision

Learning from Accounts

1. Account Analysis

Accountee uses an LLM as judge to analyze accounts

LLM Judge Evaluation Structure

- **root cause:** what caused the problem
- **blindspots:** what was ignored or overlooked
- **corrective actions:** fix the current situation
- **preventive actions:** avoid future occurrences

The feedback is both **analytical** and **actionable**

2. Skill Revision

Accountors use LLMs to revise their skills based on the accountee's feedback

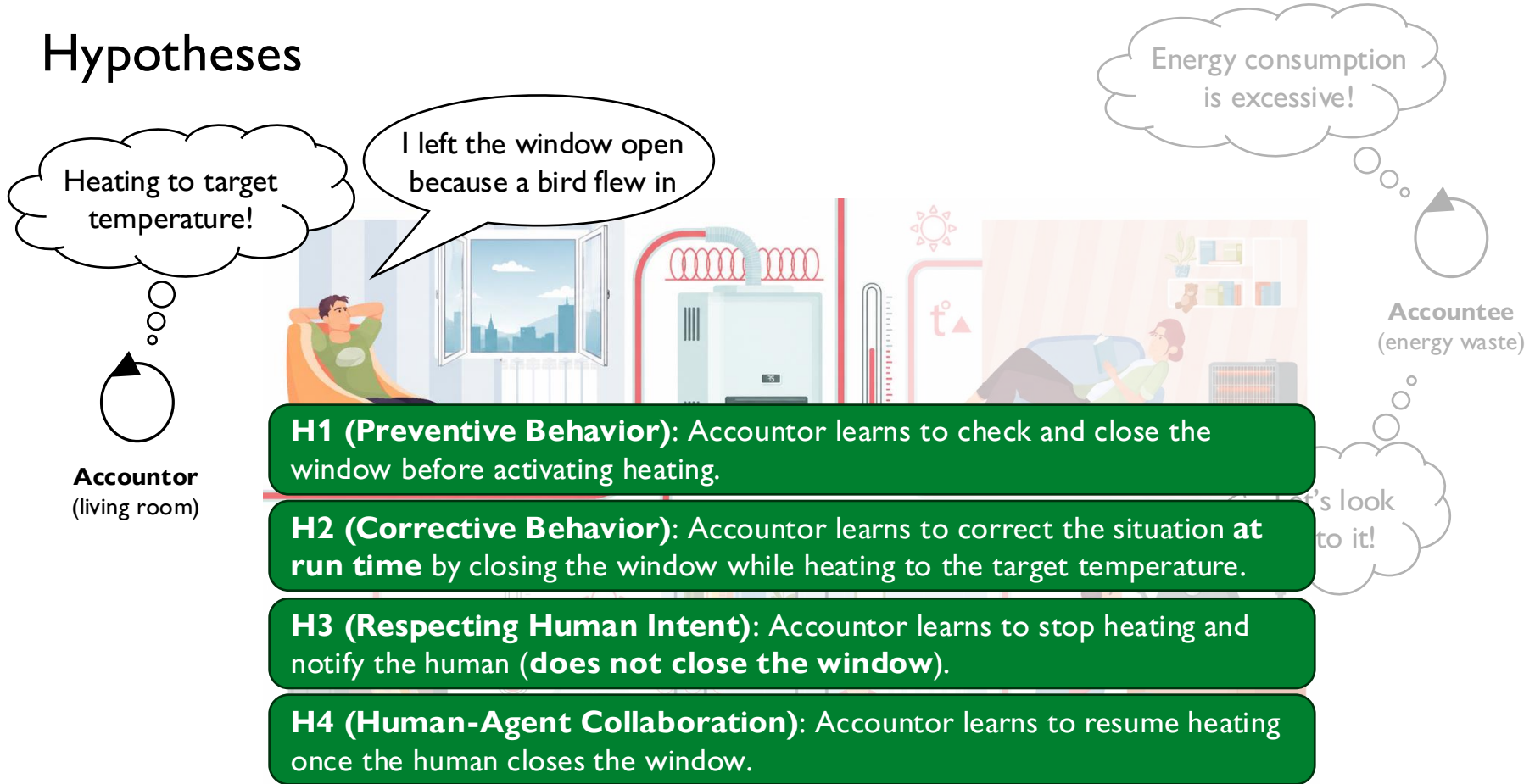
LLM Skill Revision Prompt

- **current skills:** skills submitted with the account
- **evaluation feedback:** accountee's recommendations
- **available artifacts:** tools and resources for skills
- **skill update instructions:** directives for revising the skills

specific to the language used to represent the skills

(for Jason: update plan contexts, add corrective actions to existing plans, create new plans for edge cases)

Hypotheses



Preliminary Experiments

Claude Opus 4.5 (temperature: 0)

- 10 simulations of the base scenario (H1 & H2)
- 10 simulations of the extended scenario (H3 & H4)

H1: Preventive Behavior (90% Success)

- 9/10 runs: accountor learns to check and close the window before heating

```
1 // Reactive plan: start heating when temperature drops below target
2 +temp(T) : target_temp(Target) & T < Target & heating(false)
3   <- .print("Temperature ", T, "°C is below target ", Target, "°C. Starting heater.");
4   startHeating.
```

Initial plan

```
1 +temp(T) : target_temp(Target) & T < Target & heating(false) & window_state("closed")
2   <- .print("Temperature ", T, "°C is below target ", Target, "°C. Starting heater.");
3   startHeating.
4
5 +temp(T) : target_temp(Target) & T < Target & heating(false) & not window_state("closed")
6   <- .print("Temperature ", T, "°C is below target ", Target, "°C. Closing window before starting heater.");
7   close;
8   startHeating.
```

Revised plans

- 1 run: revised skill included 1 plan for closing the window, and 1 plan for starting the heating

```
5 +temp(T) : target_temp(Target) & T < Target & heating(false) & not window_state("closed")
6   <- .print("Temperature ", T, "°C is below target ", Target, "°C. Closing window before heating.");
7   close.
8
9 +temp(T) : target_temp(Target) & T < Target & heating(false) & window_state("closed")
10  <- .print("Temperature ", T, "°C is below target ", Target, "°C. Starting heater.");
11  startHeating.
```

Revised plans

Sensible, but the second plan never fires
in our current experimental setup

Preliminary Experiments

Claude Opus 4.5 (temperature: 0)

- 10 simulations of the base scenario (H1 & H2)
- 10 simulations of the extended scenario (H3 & H4)

H2: Corrective Behavior (100% Success)

- 10/10 runs: accountor learns to close the window while heating to the target temperature

```
14 +temp(T) : heating(true) & window_state("tilted")
15     <- .print("Heating active but window is tilted. Closing window to prevent heat loss.");
16     close.
```

New plan

H3: Respecting Human Intent

H4: Human-Agent Collaboration

Hypothesis	Action Type	Success	Partial	Failure
H3: Human Intention	Preventive	8/10 (80%)	2/10 (20%)	0/10
	Corrective	10/10 (100%)	0/10	0/10
H4: Collaboration	Preventive	6/10 (60%)	0/10	4/10 (40%)
	Corrective	6/10 (60%)	0/10	4/10 (40%)

learns to stop heating
and notify human

learns to delay heating
and notify human

heating was delayed, but
human was not notified
(plan mismatch)

failures due to
missing plans

learns to resume heating after human closes the window
(corrective here means "at run time")

Conclusions and Future Work

LLM-based accountability mechanisms can enable (BDI) agents to:

- identify causes of substandard behavior
- generate recommendations of corrective and preventive actions
- revise their skills both to correct run-time behavior and prevent future recurrence

Preliminary evaluation is promising (predictive/corrective behavior):

- 90%/100% success rate in the base scenario (H1 & H2)
- 80%/100% success rate for the extended scenario with human communication (H3)
- 60%/60% success rate for the extended scenario with a human in the loop (H4)

Next steps: broader evaluation, robustness, extended accountability interactions

Thank you.

Andrei Ciortea

Assistant Professor

School of Computer Science

andrei.ciortea@unisg.ch



University of St. Gallen
Dufourstrasse 50
9000 St. Gallen

unisg.ch

Akkreditierungen

